

Inline functions

Implementing a functionality as a macro or a function is always a trade off. Hence, it is always preferable to leave the decision to the compiler itself. The keyword *inline* before the function declaration leaves the decision to GCC (depending upon whether you choose to optimise for size or for performance) when you compile the code.

Thread local storage

GCC, in conjunction with modern thread libraries, supports a special extension wherein variables could be declared local to a thread. The way to declare such variables is to use the keyword *__thread*. For example, the following statement declares an integer in a given thread context:

```
__thread unsigned int Sample;
```

This feature becomes extremely useful when writing multi-threaded applications.

Note that since 'Sample' is a thread local variable, the address of the variable is evaluated at run-time and the pointer arithmetic on it would lead to unexpected results.

Designated initialisers within arrays

Consider the following declaration:

```
enum
{
    STATE_COMPRESSION,
    STATE_EXPANSION,
    STATE_EXHAUST
};

char * states[] =
{
    "Compression",
    "Expansion",
    "Exhaust",
};
```

Hence,

```
printf("%s", elements[STATE_COMPRESSION]);
```

...will output to:

```
Compression
```

But, tomorrow if we end up swapping the position of the *STATE_COMPRESSION* and *STATE_EXHAUST* in the enumeration, the names of the objects will change and the output of the same *print* statement will be:

```
Exhaust
```

Here is how we can write the same with GCC:

```
char * states[] =
{
    [STATE_COMPRESSION] = "Compression",
    [STATE_EXPANSION] = "Expansion",
    [STATE_EXHAUST] = "Exhaust",
};
```

And come what may, the *STATE_COMPRESSION* is *Compression* and *STATE_EXHAUST* is *Exhaust*.

Case range

Short programs are always more readable and smarter. Consider the following code snippet:

```
switch(value)
{
    case 1:
    case 2:
    case 3:
    case 4:
        printf("First four");
        break;
    case 5:
    case 6:
    case 7:
        printf("Till seven");
        break;
    case 8:
    case 9:
        printf("Big bros");
        break;
    default:
        printf("Too much.....!!");
}
```

Well, there's no need to write so much. You can simply get away with:

```
switch(value)
{
    case 1 ... 4:
        printf("First four");
        break;
    case 5 ... 7:
        printf("Till seven");
        break;
    case 8 ... 9:
        printf("Big bros");
        break;
    default:
        printf("Thanks GCC.....!!");
}
```